

CSEN 296B-2 | Week 5.2 Lab Submission

Requirements-to-UML Traceability Matrix for AstraNotes

Student Name: Atishay Jain | **Date:** May 4, 2026 | **Project:** AstraNotes | **Technical Path:** Python

I picked the eight highest-risk requirements from *planning/refined-requirements.md*: one validation gate (FR-01), two cross-cutting workflows (FR-04, FR-05), one read-side feature (FR-07), one architectural rule (NFR-02), the privacy backbone (SPR-01), error handling (SPR-02), and the dependency governance rule (SPR-04). NFR-01 and NFR-03 are runtime-performance assertions and were skipped on purpose; they belong to a Sprint 1 benchmark harness, not a UML view.

Traceability Matrix

ID	Requirement	Class / Object Evidence	Use Case / Activity Evidence	Deployment Evidence	Status	Gap Note
FR-01	Create a note with a non-empty title and an optional body. UUID and timestamps are auto-assigned. Whitespace-only titles are rejected.	Class diagram declares Note , ValidationLayer.validate(note) , and NoteManager.create_note(...) . Object diagram shows three Note instances with valid titles and timestamps.	UC1 'Create note (FR-01)'. Activity diagram has the ValidationLayer.validate decision diamond branching to 'Raise ValidationError - FR-01'.	services/ has ValidationLayer and NoteManager; models/ has Note.	Fully Traced	None.
FR-04	Toggle a note's is_private flag. When True, body is encrypted before write. When toggled back to False, body is re-saved as plaintext. If decryption fails during toggle, the operation fails and the note stays encrypted.	Class diagram declares NoteManager.toggle_privacy(note_id) and PrivacyService.encrypt/decrypt . Object diagram's note2 shows the encrypted state (body = b'gAAAAAB...').	UC4 'Toggle note privacy (FR-04, SPR-01)'. The activity diagram covers the encrypt path of create-private; the toggle-back-to-plaintext path and the decrypt-failure path share the same Save and SPR-02 surface steps but are not drawn as a separate flow.	PrivacyService in services/; Fernet key store in FS.	Partially Traced	All structural pieces are present; missing piece is a dedicated activity flow for the toggle-back and decrypt-fail paths. Planned for Sprint 1 prerequisite diagrams.
FR-05	Persist notes one file per note. On startup, valid files load and any unreadable file is skipped with an error logged. The data directory is created on first launch if it does not exist.	Class diagram declares JsonFileRepository(data_dir: Path) with save/get/list_all/update/delete. Object diagram's Disk subgraph shows three {uuid}.json files.	UC7 'Persist and restore on startup (FR-05, FR-06, NFR-03)'. Activity diagram shows JsonFileRepository writing data_dir/{id}.json plus the OSError-to-PersistenceError translation.	data/ filesystem node tagged FR-05, FR-06; JsonFileRepository in repositories/.	Partially Traced	Class, object, and use case views cover the rule. The startup-time load loop (with skip-and-log on a bad file and first-launch directory creation) is not drawn as an activity. Planned for Sprint 1 prerequisite diagrams.
FR-07	Keyword search across titles and bodies; case-insensitive, plain string matching. Successfully decrypted private notes are included; notes that fail to decrypt are excluded with an error logged. Empty collection returns an empty list.	Class diagram declares NoteManager.search_notes(keyword: str) list~Note~ . PrivacyService.decrypt is on the same diagram, so the decrypt-then-match coupling is structurally available.	UC6 'Search notes by keyword (FR-07)'. No activity diagram yet for the search workflow itself.	NoteManager and PrivacyService co-located in services/.	Weakly Traced	Method signature and use case are present; the decrypt-then-match loop and the empty-collection branch need their own activity diagram. Planned for Sprint 1 prerequisite diagrams.

ID	Requirement	Class / Object Evidence	Use Case / Activity Evidence	Deployment Evidence	Status	Gap Note
NFR-02	Architecture enforces separation between NoteManager, NoteRepository / JsonFileRepository, and PrivacyService so each can be unit-tested independently.	Class diagram shows NoteRepository < -- JsonFileRepository realization and three explicit aggregation edges from NoteManager to NoteRepository, PrivacyService, and ValidationLayer (the dependency-injection seam).	Not directly visible (NFR-02 is structural, not behavioral).	Each layer lives in its own folder (models/, repositories/, services/, cli/) inside one process.	Fully Traced	None.
SPR-01	Private bodies encrypted with Fernet before any disk write. Key is never stored in source code or in the notes directory. Key management strategy documented in the ADL before Sprint 1.	Class diagram declares PrivacyService(fernet, encrypt, decrypt, generate_key) . Object diagram's note2 body is shown as Fernet ciphertext.	UC4 (tagged SPR-01). Activity diagram's Encrypt step explicitly precedes JsonFileRepository.save when is_private is true.	Fernet key store node is separate from data/ and labelled 'ADR pending in Sprint 1 - SPR-01' so the open key-storage decision stays visible.	Fully Traced	None.
SPR-02	All storage exceptions are caught and translated to user-level messages. No file paths, stack traces, or internal state is exposed.	Class diagram declares AstraNotesError parent with NoteNotFoundError, PersistenceError, ValidationError subclasses. Explicit ..> raises edges from JsonFileRepository, PrivacyService, and ValidationLayer.	UC8 'Surface storage errors safely (SPR-02, SPR-03)' with <<include>> arrows from every write-side use case. Activity diagram surfaces a user-friendly message on both the validation failure branch and the persistence failure branch.	Exception module models/exceptions: listed in deployment diagram.	Fully Traced	None.
SPR-04	Third-party libraries are recorded with pinned version, purpose comment, and confirmed permissive license (MIT, Apache 2.0, BSD).	Not visible in class or object diagram (this is a build-time governance rule).	Not visible in use case or activity diagram.	Deployment diagram's Dops subgraph lists cryptography 44.0.0 (Fernet) and pytest 8.3.4 dev only , with the SPR-04 tag inline.	Partially Traced	Versions and purpose are visible; license strings are not. I will add the license label to each Dops node in the deployment diagram before Sprint 1.

Traceability Metrics

Metric	Value
Total requirements reviewed	8
Fully Traced	4 (FR-01, NFR-02, SPR-01, SPR-02)
Partially Traced	3 (FR-04, FR-05, SPR-04)
Weakly Traced	1 (FR-07)
Not Traced	0
Major UML elements without a clear requirement reason	2

Orphaned UML elements

Two design elements appear in the UML package without a clear requirement reason behind them:

1. The 'User Workstation - macOS / Linux / Windows, Python 3.10+' node on the deployment diagram. The cross-platform OS list and the specific minimum Python version are not pinned by any FR / NFR / SPR. They are reasonable Python defaults, but they imply portability coverage I have not committed to. I will either add a portability NFR or trim the label to a generic 'Local workstation' before Sprint 1.

2. The `generate_key()` method on `PrivacyService`. It is useful tooling but SPR-01 only requires that the key not live in source code or the data directory; it does not require an in-process key-generation API. The Sprint 1 SPR-01 ADR will either tie this method to a concrete requirement or move it into a dev-only script.

Gap Analysis

All eight requirements have at least structural and use-case coverage. The gaps are concentrated in the activity diagrams: today there is one (create-private), and three more are needed before Sprint 1 implementation work begins.

1. FR-07 (search) is the largest gap. The search workflow is the most subtle behavior in the system because it cuts across encryption, error logging, and an empty-collection short-circuit. Today it is represented only by a method signature and a use-case oval. A search activity diagram is needed before *NoteManager.search_notes* is written.

2. FR-04 (toggle privacy) needs the toggle-back and decryption-failure branches drawn. The pieces are all on the class diagram, but the only activity flow is for create-private. Drawing the inverse path and the decrypt-failure path will lock down the failure semantics before code.

3. FR-05 (persistence) needs a startup-flow activity diagram. Save and OSError translation are covered, but the load loop with skip-and-log on a bad file and the first-launch directory creation are only described in prose, not modeled.

A fourth, lower-priority refinement is **SPR-04**: each third-party library node on the deployment diagram should carry its license string (cryptography 44.0.0 - Apache 2.0 / BSD, pytest 8.3.4 - MIT) so the license-confirmation rule shows up in UML as well as in pyproject.toml.

Planned Refinements Before Sprint 1

Each Partially / Weakly Traced requirement above has a concrete plan to close it before Week 6.1 implementation kickoff:

Requirement	Status today	Action before Sprint 1
FR-04 (toggle privacy)	Partially Traced	Extend the create-private activity diagram with the toggle-back-to-plaintext path and the decrypt-failure-keeps-encrypted path.
FR-05 (persistence)	Partially Traced	Add a startup-flow activity diagram covering load-loop, skip-and-log on a bad file, and first-launch directory creation.
FR-07 (search)	Weakly Traced	Add a search activity diagram showing decrypt-then-match, skip-and-log on decrypt failure, and empty-collection short-circuit.
SPR-04 (deps governance)	Partially Traced	Add a license string to each library node in the deployment diagram (cryptography 44.0.0 - Apache 2.0 / BSD, pytest 8.3.4 - MIT).

How AI Helped Build This Matrix

I used AI as a drafting and consistency-checking helper, not as the source of truth for the status labels.

What AI did. Scaffolded the table from *planning/refined-requirements.md* and the requirements coverage list at the bottom of *docs/uml/design-rationale.md*, normalized the requirement wording to match the refined baseline, and proposed a first-pass status assignment based on whether each requirement was named in any single diagram. It also enumerated UML elements as candidates for the no-clear-requirement-reason list.

What I changed. AI's first pass marked FR-04 and FR-05 as Fully Traced because each had a use case plus a class-diagram method. After re-reading the refined baseline I downgraded both to Partially Traced, since the toggle-back, decrypt-failure, load-loop, and first-launch-directory behaviors are required by the requirement wording but not yet drawn as activity flows. FR-07 went from Partially to Weakly Traced for the same reason - a method signature is not behavioral evidence for the algorithmically subtlest feature in the system. Every gap note was rewritten to name a specific missing artifact and to point to the planned refinement table.

What I rejected. AI suggested adding NFR-01 and NFR-03 to the matrix and labelling them Not Traced. I rejected that: those are runtime-performance assertions, not design requirements, and the design rationale already documents that they are deferred to a Sprint 1 benchmark harness. Marking them Not Traced would inflate the gap count without exposing a real design problem. AI also suggested treating NoteManager as an orphan because it is not yet implemented; I rejected that for the same reason given in the Week 4.2 reflection - the class diagram describes the target architecture, and NoteManager is the orchestrator the other views depend on.

The final mapping, status labels, metrics, gap analysis, and planned-refinement actions are my judgment.